TECH DEBT

ELEVATED

DOCUMENTATION DESERT

A critical integration with a payment processor was never documented. The engineer who built it left six months ago. It just started failing in a new way. Nobody on the current team knows what it does, how it works, or where to even start debugging.

Documentation debt is the most invisible form of technical debt — it generates no compiler warnings, no test failures, and no alerts until the moment someone needs the knowledge. The DevOps philosophy treats documentation as a first-class engineering artefact: runbooks live alongside code in version control; Architecture Decision Records (ADRs) capture not just what was decided but why; OpenAPI / AsyncAPI specifications auto-generate and stay current; and READMEs are part of the definition of done. The concept of 'docs-as-code' — applying the same review, version control, and CI practices to documentation as to code — keeps documentation from drifting. Platforms like Backstage provide internal developer portals that centralise service documentation and ownership, so the question 'who owns this and how does it work?' always has an answer.

Undocumented systems are technical debt you discover at the worst possible time — when something breaks and the author is gone.

TEAM DISCUSSION

Could a new engineer operate your most critical systems from your documentation alone, without asking anyone for help?